

MSAS – Assignment #1: Simulation

Andrea D'Uva, 942644

1 Implicit equations

Exercise 1

Given the function $f(x) = \cos x - x$, guess a and b such that $f(a)f(b) < 0$. Find the zero(s) of f in $[a, b]$ with 1) the bisection method, 2) the secant method, 3) the regula falsi method. All solutions must be calculated with 8-digit accuracy. Which method requires the least number of function evaluations? Report the computational time required by each method.

The first thing to do when trying to solve an implicit equation is to try and represent graphically its behaviour in a significant domain. For the given function such broad graph showed that only one zero exists.

After being sure of that one can go for a more detailed graphical study of the behaviour close to the zero as showed in Figure 1; in this case the implicit function is "well behaved", meaning well conditioned and well posed, so we can safely apply our numerical methods.

Setting a tolerance of 10^{-8} for each of them delivers the result $x_0 = 0.73908513$.

Rigorously speaking each of the three methods requires only one function evaluation $f(x_k)$ per iteration; however they differ in how many times such value is used at each step: the bisection method uses it only once, the secant method uses it twice while the regula falsi technique uses it three times.

This is reflected in the total number of iterations required by the methods to converge to the solution. With all of them starting from the interval $a = -5, b = 5$, they take 29, 7 and 8 iterations, respectively, to reach the zero.

These results don't come as a surprise: using a higher number of function evaluations delivers a better convergence rate, even if the algorithm becomes slightly more complex. Finally we can compare also the computational time required by each method. To ensure a standard comparison each method has been run 7 times before the measure.

The higher number of iterations done is reflected in the computational time: the bisection method took $3.01 \cdot 10^{-6}s$, while the secant method and the regula falsi method took $1.23 \cdot 10^{-6}$ and $1.11 \cdot 10^{-6}$.

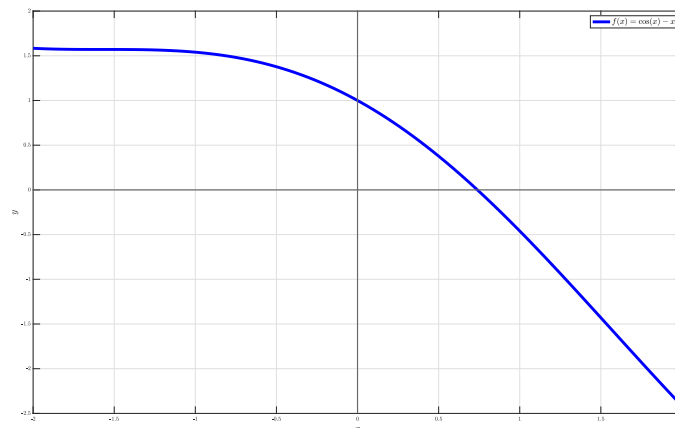


Figure 1: function behaviour close to the zero

Exercise 2

Let $\mathbf{f}$ be a two-dimensional vector-valued function $\mathbf{f}(\mathbf{x}) = (x_1^2 - x_1 - x_2, x_1^2/16 + x_2^2 - 1)^\top$, where $\mathbf{x} = (x_1, x_2)^\top$. Find the zero(s) of $\mathbf{f}$ by using Newton's method with $\partial\mathbf{f}/\partial\mathbf{x}$ 1) computed analytically, and 2) estimated through finite differences. Which version is more accurate?

As in the previous exercise the first step is to graphically represent the function behaviour. In this case, being the function vectorial with two independent variable, the choice made was to draw the contour lines for the two outputs and look for one or more intersection(s) as shown in Figure 2.

The next step is studying the behaviour of the derivatives around the zeros to ensure that the implicit function is well posed and well conditioned. Taking a look at the Jacobian matrix $J = \begin{bmatrix} 2x_1 - 1, -1 \\ \frac{x_1}{8}, 2x_2 \end{bmatrix}$ we can immediately be sure that we will not encounter any issue in an appropriate neighbourhood of the two zeros.

The results provided by the Newton's method using the analytical Jacobian and its approximation through both forward and centered finite differences are $x_1^{zero} = \begin{bmatrix} 1.581 \\ 0.918 \end{bmatrix}$ and

$x_2^{zero} = \begin{bmatrix} -0.613 \\ 0.988 \end{bmatrix}$. Only one value has been reported because the difference between the three results is in the order of 10^{-9} , even though the tolerance set was 10^{-7} . Being both approximations of the derivatives so accurate, even beyond the prescribed constraints, is due to the Jacobian matrix having only polynomials of order 1, which both forward and centered differences can catch exactly.

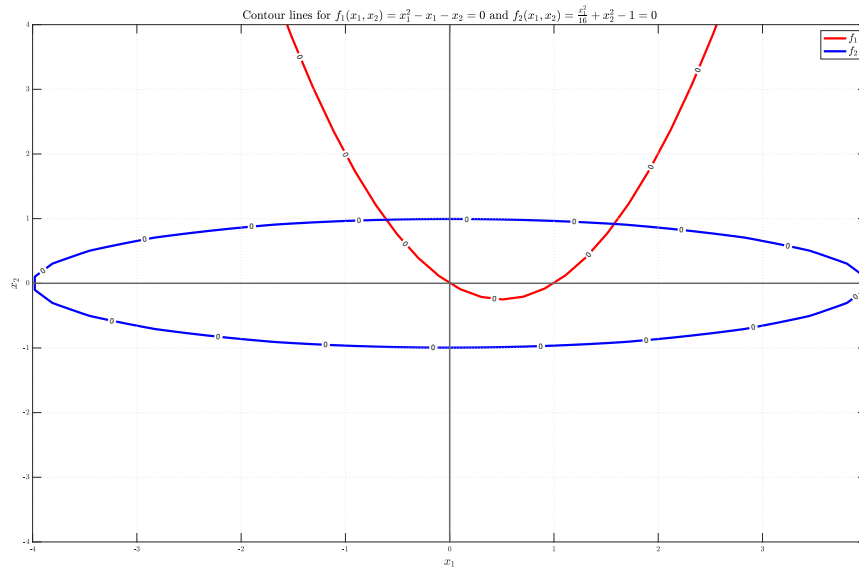


Figure 2: contour lines

2 Numerical solution of ODE

Exercise 3

The Initial Value Problem $\dot{x} = x - t^2 + 1$, $x(0) = \frac{1}{2}$, has analytic solution $x(t) = t^2 + 2t + 1 - \frac{1}{2}e^t$. 1) Implement a general-purpose, fixed-step Heun's method (RK2); 2) Solve the IVP in $t \in [0, 2]$ for $h_1 = 0.5$, $h_2 = 0.2$, $h_3 = 0.05$, $h_4 = 0.01$ and compare the numerical vs the analytical solution; 3) Repeat points 1)–2) with RK4; 4) Trade off between CPU time & integration error.

The results provided by numerical integration of the given IVP through general purpose RK2 and RK4 methods are reported in Figures 3 and 4. In both cases the solution for $h_4 = 0.01$ was not drawn as it would have almost resembled the analytical solution, being the integration error extremely limited. It is worth noticing that the two methods have been fully capable of retaining the unstable nature of the differential equation, which appears clearly when looking at the exact solution. Moreover, as expected, a decrease of the step-size h results in a lower integration error. However the different order of the two methods implies that reaching a given integration error makes the lower-order scheme use a step-size significantly higher than the higher-order scheme.

This is evident by comparing the CPU time against the integration error for the two methods, where CPU time is strictly related to the h amplitude, as shown in Figure 5.

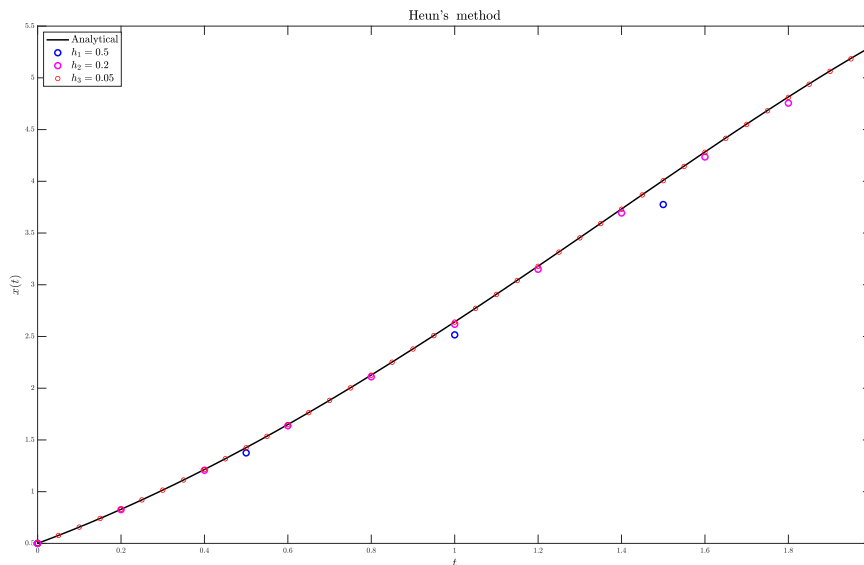


Figure 3: Heun's method

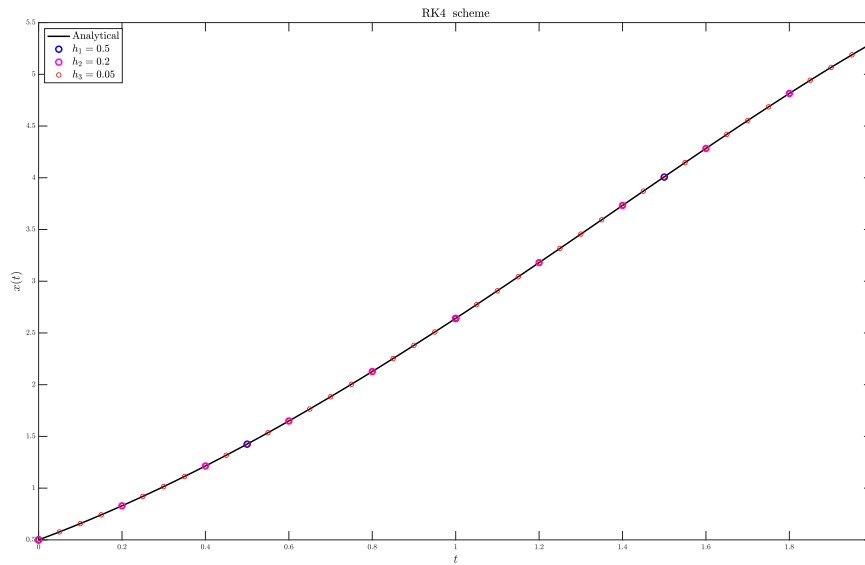


Figure 4: RK4 method

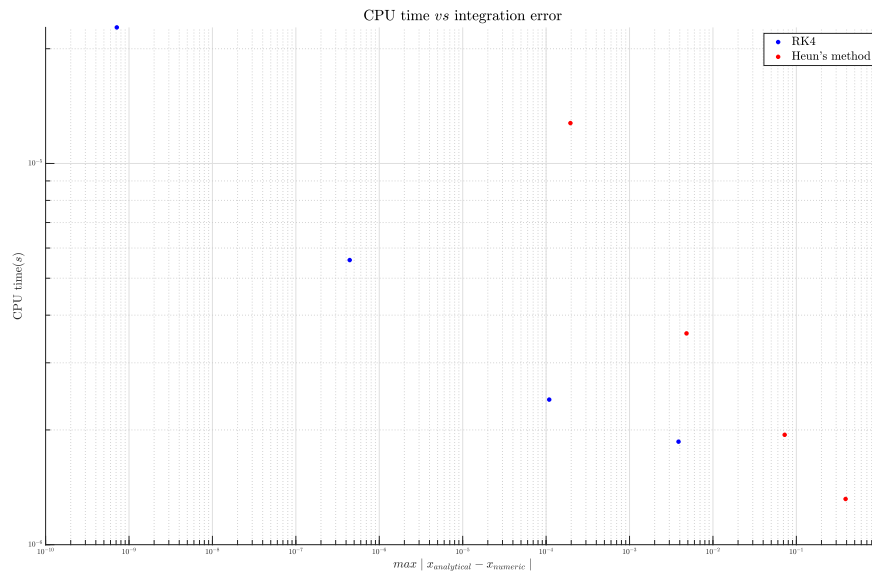


Figure 5: CPU time *vs* integration error

Exercise 4

Let $\dot{\mathbf{x}} = A(\alpha)\mathbf{x}$ be a two-dimensional system with $A(\alpha) = [0, 1; -1, 2\cos\alpha]$. Notice that $A(\alpha)$ has a pair of complex conjugate eigenvalues on the unit circle; α denotes the angle from the $\text{Re}\{\lambda\}$ -axis. 1) Write the operator $F_{\text{RK2}}(h, \alpha)$ that maps $\mathbf{x}_k$ into $\mathbf{x}_{k+1}$, namely $\mathbf{x}_{k+1} = F_{\text{RK2}}(h, \alpha) \mathbf{x}_k$. 2) With $\alpha = \pi$, solve the problem “Find $h \geq 0$ s.t. $\max(|\text{eig}(F(h, \alpha))|) = 1$ ”. 3) Repeat point 2) for $\alpha \in [0, \pi]$ and draw the solutions in the (h, λ) -plane. 4) Repeat points 1)–3) with RK4 and represent the points $\{h_i \lambda\}$ of Exercise 3 with $t = 0$. What can you say?

The two operators $F(h, \alpha)$ are found by substituting the LTI ode inside the expressions of

the two numerical schemes. By doing so one can find:

$$F_{\text{RK2}}(h, \alpha) = I + hA(\alpha) + \frac{1}{2}(hA(\alpha))^2$$

$$F_{\text{RK4}}(h, \alpha) = I + hA(\alpha) + \frac{1}{2}(hA(\alpha))^2 + \frac{1}{6}(hA(\alpha))^3 + \frac{1}{24}(hA(\alpha))^4$$

These expressions clearly resembles a Taylor Series Expansion: it is not a coincidence since the idea of the fixed-step explicit RK schemes is to actually approximate $f(t^* + h)$ by means of a TSE up to different orders. The stability regions of the two methods are represented by Figure 6, where a scheme is stable if the discrete eigenvalues $h\lambda$ fall inside the drawn regions. A remark on exercise 3 can now be made, even if not strictly rigorous as the differential equation didn't represent a LTI system: both methods were able to retain the unstable nature of the solution because for all step-sizes h the discrete eigenvalues fall in the unstable region. Finally an observation about the proposed method to find the locus of $(h\lambda)$ that make the schemes stable: as mostly evident from the RK4 edge lines, a non-smooth behaviour occurs because of the implicit equation in h becomes bad-posed for values of $\alpha < 85^\circ$. See Figure7 for a graphical understanding. A smooth curve can instead be obtained by referring to a different expression of the problem. Representing a scheme as $\mathbf{x}_{k+1} = R(h\lambda)\mathbf{x}_k$, its stability region is given by the values $z = h\lambda \in C : |R(h\lambda)| < 1$ being $R(h\lambda) = \sum_{s=1}^n \frac{1}{s!}(h\lambda)^s$ with $n = 1, 2, 3, 4$ depending on the order of the RK scheme. For more details the reference is the book by A.Quarteroni, R.Sacco "Matematica Numerica".

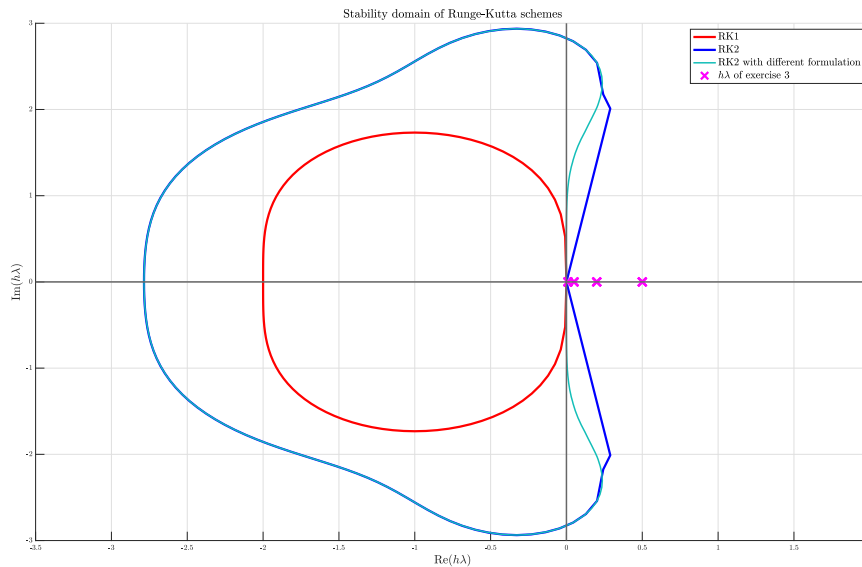


Figure 6: Stability domains

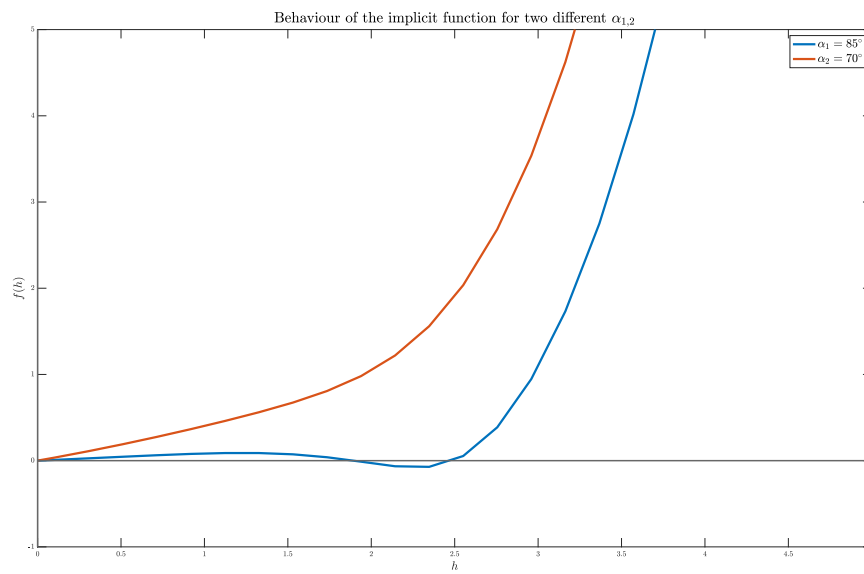


Figure 7: Implicit equations for different α

Exercise 5

Consider the IVP $\dot{\mathbf{x}} = A(\alpha)\mathbf{x}$, $\mathbf{x}(0) = [1, 1]^T$, to be integrated in $t \in [0, 1]$. 1) Take $\alpha \in [0, 2\pi]$ and solve the problem “Find $h \geq 0$ s.t. $\|\mathbf{x}_{\text{an}}(1) - \mathbf{x}_{\text{RK1}}(1)\|_{\infty} = \text{tol}$ ”, where $\mathbf{x}_{\text{an}}(1)$ and $\mathbf{x}_{\text{RK1}}(1)$ are the analytical and the numerical solution (with RK1) at the final time, respectively, and $\text{tol} = \{10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}\}$. 2) Plot the five locus of solutions in the $(h\lambda)$ -plane; plot also the function evaluations vs tol for $\alpha = \pi$. 3) Repeat points 1)–2) for RK2 and RK4.

The main issue of this problem was to find proper algorithms and initial conditions able to provide a good solution in a reasonable time. After a purely trial and error procedure such issue were solved and the problem could be solved. The main computational problem however involved using a proper function for each numerical scheme: the general purpose ones weren't suitable. Instead writing their last values as powers of their own forward linear operator, expressed in terms of the matrix $A(\alpha)$, proved to be a much better choice. It is worth noticing that since RK1, RK2 and RK4 require 1, 2 and 4 function evaluations at each step respectively, the result shown by Figure 11 was expected. For a given tolerance on the integration error a higher-order scheme will naturally need a lower step-size h which will globally result in a lower number of total function evaluations. As a consequence in the logarithmic chart the sequence of points appear to be placed along straight lines with slopes of $-1, -2, -4$. This is not a coincidence for the reason explained above: these slopes are exactly the number of function evaluations that each scheme uses, and indeed matches also their order.

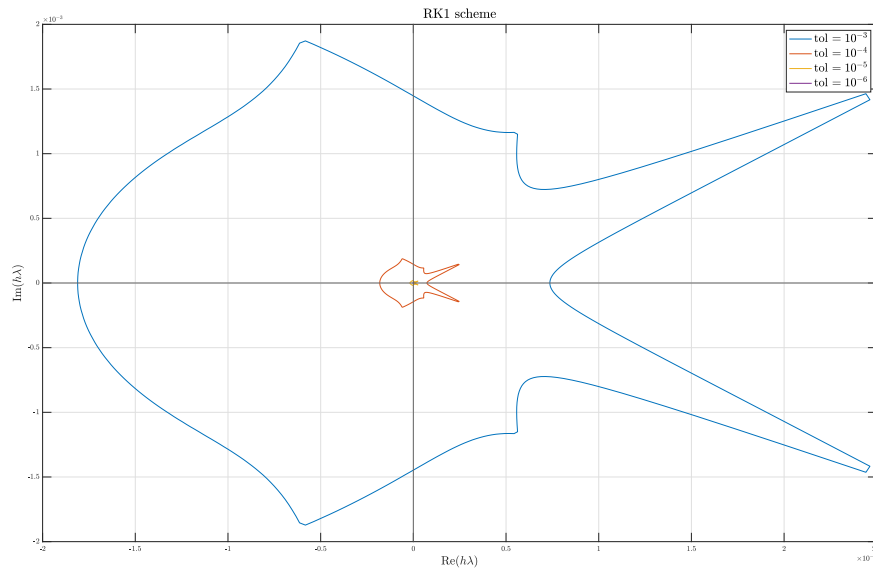


Figure 8

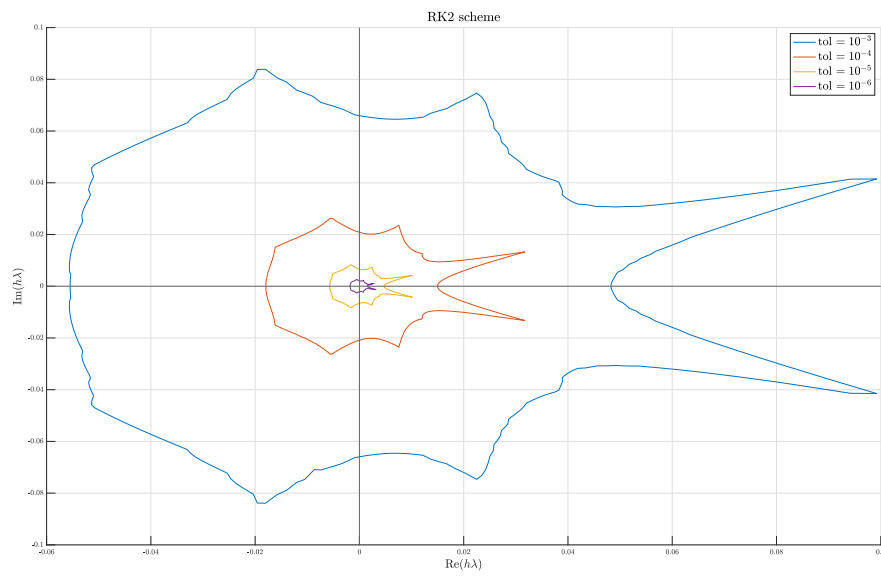


Figure 9

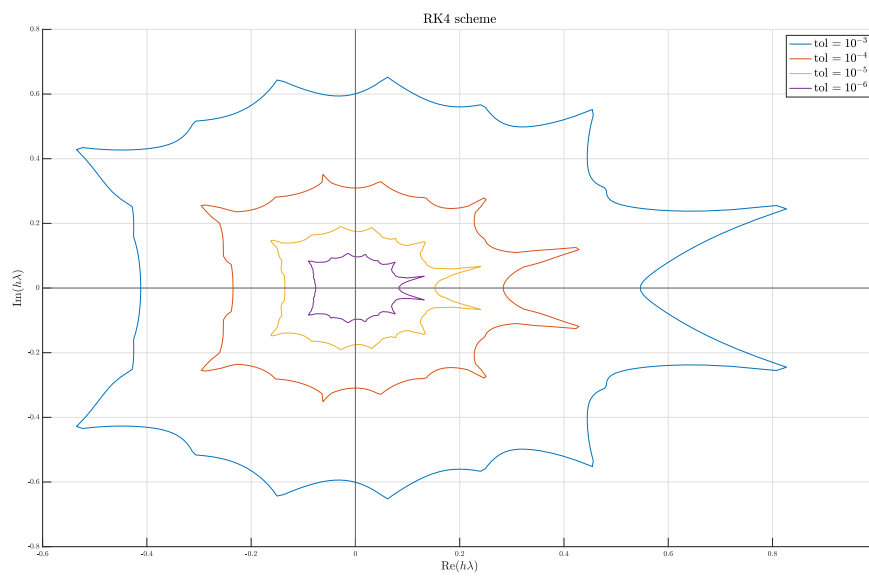


Figure 10

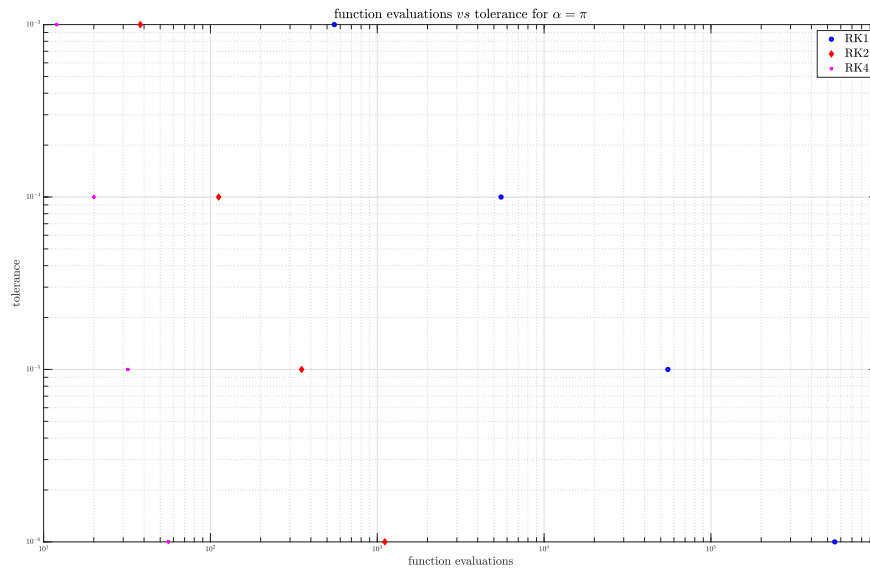


Figure 11

Exercise 6

Consider the backinterpolation method $BI_{0.4}$. 1) Derive the expression of the linear operator $B_{BI_{0.4}}(h, \alpha)$ such that $\mathbf{x}_{k+1} = B_{BI_{0.4}}(h, \alpha)\mathbf{x}_k$. 2) Following the approach of point 3) in Exercise 5, draw the stability domain of $BI_{0.4}$ in the $(h\lambda)$ -plane. 3) Derive the domain of numerical stability of $BI_{2\theta}$ for the values of $\theta = [0.1, 0.3, 0.7, 0.9]$.

To compute the linear operator $B_{BI_{2\theta}}(h, \alpha)$ it is necessary to plug the linear differential equation of the LTI system into a backinterpolation method that a $RK2(\theta h)$ step forward and a $RK2(-h(1 - \theta))$ step backward, making it then an implicit scheme. After substitutions and rearrangement the operator is found to be:

$$B_{BI_{2\theta}}(h, \alpha) = (I - (1 - \theta)hA(\alpha) + \frac{1}{2}(1 - \theta)^2(hA(\alpha))^2)^{-1}(I + \theta hA(\alpha) + \frac{1}{2}(\theta hA(\alpha))^2)$$

Then retrieving the stability domains in this case is quite straightforward and the only issue is to remember that in order to correctly get the reversed regions ($\theta = 0.7$ and $\theta = 0.9$) we should start from $\alpha = \pi$ instead of $\alpha = 0$ as for the other curves. For the schemes in the right-hand semi-plane the stability region is outside the curves while it is the opposite for the methods in the left-hand semi-plane.

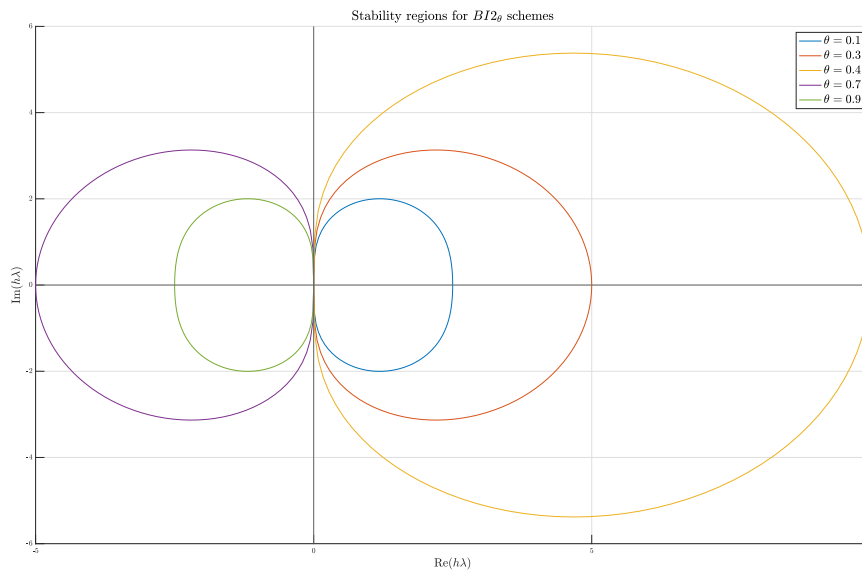


Figure 12: Stability domains of $B_{BI_{2\theta}}$

Exercise 7

Consider the IVP $\dot{\mathbf{x}} = B\mathbf{x}$ with $B = [-180.5, 219.5; 179.5, -220.5]$ and $\mathbf{x}(0) = [1, 1]^T$ to be integrated in $t \in [0, 5]$. Notice that $\mathbf{x}(t) = e^{Bt}\mathbf{x}(0)$. 1) Solve the IVP using RK4 with $h = 0.1$; 2) Repeat point 1) using BI2_{0.1}; 3) Compare the numerical results in points 1) and 2) against the analytic solution; 4) Compute the eigenvalues associated to the IVP and represent them on the $(h\lambda)$ -plane both for RK4 and BI2_{0.1}; 5) Discuss the results.

In this case the B matrix has two eigenvalues: $\lambda_1 = -1$ and $\lambda_2 = -400$ which make the system to be stable. However while BI2_{0.1} can catch this behaviour nicely as shown in Figure 13 and 14, the RK4 scheme skyrockets, reaching values up to the order of 10^{248} , and then its representation isn’t useful at all. Such an issue is caused by the system being stiff: as depicted by Figure 15 for $h = 0.1$ RK4 can’t catch both the eigenvalues, thus becoming unstable. On the contrary, being BI2_{0.1} an A-stable scheme, it is fully capable of preserving the stable nature of the system no matter of the step-size.

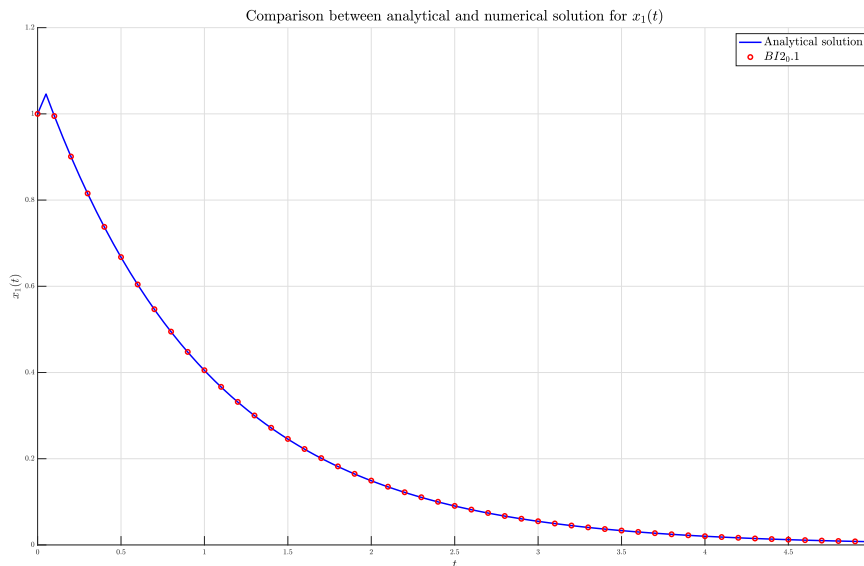


Figure 13: $x_1(t)$ behaviour

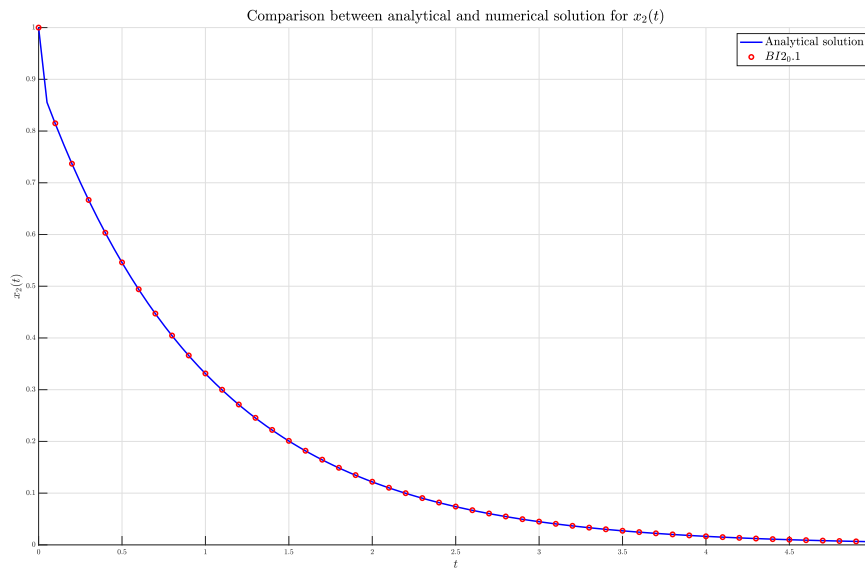


Figure 14: $x_2(t)$ behaviour

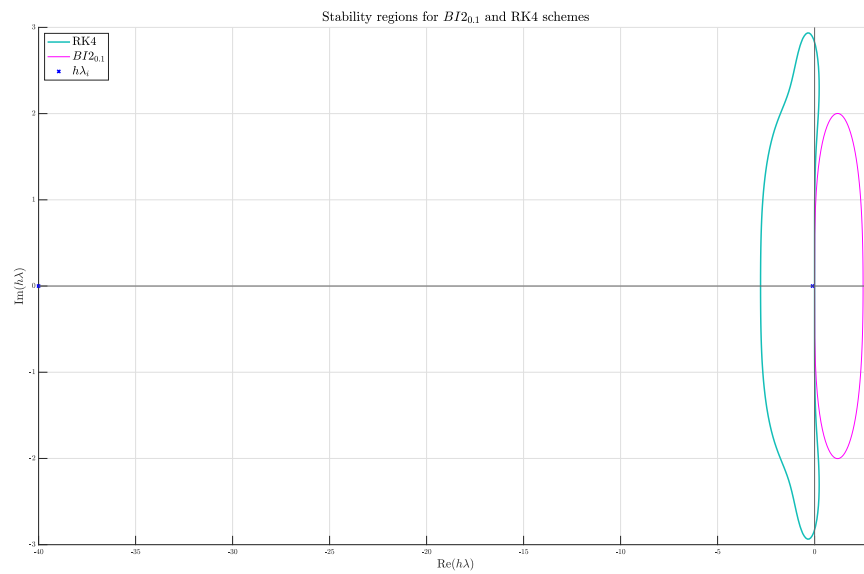


Figure 15: Stability domains of the two schemes and relative eigenvectors